

Vedant Desai

(248) 704 – 4852 | vedantde@umich.edu | github.com/Verdent06 | linkedin.com/in/vedantde06

EDUCATION

University of Michigan

B.S. in Computer Science and Economics

Expected May 2028

Ann Arbor, MI

- **GPA:** 3.66 / 4.0
- **Coursework:** Data Structures & Algorithms, Intro to Statistics and Data Analysis, Microeconomics, Macroeconomics, Discrete Mathematics, Calculus III, Physics (Mechanics)

EXPERIENCE

Michigan Data Consulting (MDC)

Jan 2026 – May 2026

Data Engineer — Michigan Campaign Finance Network

Ann Arbor, MI

- Delivered a production Flask REST API on AWS EC2 as the sole engineer on a 5-month MCFN contract, wiring ingested data and PAC rankings into the nonprofit's public-facing research workflow.
- Replaced manual Michigan campaign-finance research — portal searches capped by the Bureau of Elections, irregular Excel exports, hand normalization at ~2 hours per committee — with a Requests + Pandas ETL that ingests filings directly, eliminating ~800 hours of manual pulls across 400 tracked PACs.

Granular Synthesizer Plugin | C++, JUCE, CMake

github.com/Verdent06/granular-synth

Real-time audio DSP plugin built from scratch in C++/JUCE — sine oscillator through full granular engine with lock-free threading and release-ready VST3/AU binaries

- Enforced the audio thread's zero-allocation constraint by pre-allocating a `MemoryPool<Grain, 64>` slab per voice at plugin startup — a fixed free-list that acquires and releases grain slots without touching the heap — so `processBlock()` never calls `new` or `delete` after `prepareToPlay()` completes.
- Configured CMake to compile VST3 and AU bundles from a single JUCE codebase — targeting a macOS universal binary (arm64 + x86_64) for native Apple Silicon and Intel — and audited every `processBlock()` path against a real-time safety checklist confirming zero heap allocations and zero lock acquisitions before release builds.
- Wired UI-to-audio delivery through a hand-rolled SPSC FIFO (64-slot fixed array, atomic head/tail with acquire/release ordering) so slider changes reach the audio thread without a mutex — and routed WAV handoff via atomic shared_ptr swap so the UI thread never blocks `processBlock()`.
- Built the granular engine around a fractional-accumulator scheduler that fires grains at $\text{density} \times (1 - \text{overlap} \times 0.5)$ effective rate — each grain reads a random WAV offset at $2^{(\text{semitones}/12)}$ playback speed, windowed by a pre-computed 8,192-entry Gaussian LUT to eliminate click artifacts at grain edges, with all grain memory drawn from a per-voice pre-allocated pool.

CaseStudyPrep.AI

Dec 2025 – May 2026

Software Engineer Co-op (Voice AI)

Remote

- Moved audio processing off the UI thread into a Web Worker with an async stream handoff, reducing main-thread blocking time to under 5ms and keeping the real-time audio visualizer at a smooth 60 FPS during active inference.
- Eliminated a 27% audio upload failure rate with fault-tolerant RxJS logic that detects expired S3 presigned URLs, regenerates them mid-flight, and negotiates MIME types for WAV files Angular silently rejected.

PROJECTS

SignalWeaver | Python, FastAPI, Docker, GitHub Actions

github.com/Verdent06/SignalWeaver

Multi-signal financial research platform combining fundamentals, macro indicators, and news sentiment (research assistant; not investment advice)

- Served composite financial-research scores through async REST endpoints (FastAPI) wrapping fundamentals, MPNet sentiment, and regression logic, instrumented end-to-end at 9.1s p50 / 15.2s p99 across 90 successful runs on 90 tickers.
- Containerized the stack with Docker Compose (API + Postgres/pgvector + nginx frontend) and a GitHub Actions CI pipeline (frontend build, pytest, API image build on main).

TECHNICAL SKILLS

Languages C++, Python, TypeScript, SQL, HTML/CSS
Frameworks JUCE, Flask, FastAPI, Angular, RxJS
Cloud & Infrastructure AWS (EC2, S3), Docker, Git, GitHub Actions